

HOSPITAL ASSET MANAGEMENT AND TRACKING PORTAL

MINI PROJECT REPORT

submitted by

TESSA JOSE (AJC23CS199)

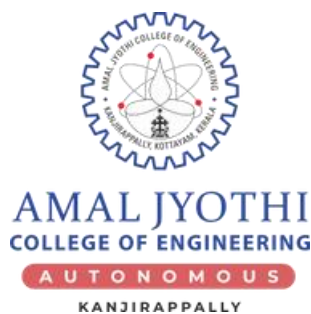
under the guidance of

Ms. HITHA P S

Assistant Professor

to

the APJ Abdul Kalam Technological University
in partial fulfillment of the requirement for the award of the Degree of
Bachelor of Technology
in
Computer Science and Engineering



Department of Computer Science and Engineering

Amal Jyothi College of Engineering
(Autonomous)

Kanjirappally-686518

March 2026

DECLARATION

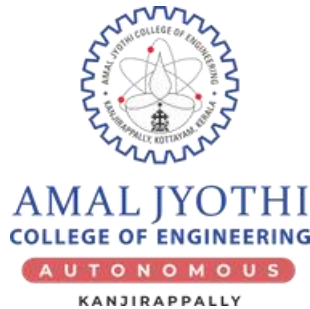
I undersigned hereby declare that the project report “**HOSPITAL ASSET MANAGEMENT AND TRACKING PORTAL**”, submitted for partial fulfillment of the requirements for the award of degree of Bachelor of Technology of the APJ Abdul Kalam Technological University, Kerala is a Bonafide work done by me under the supervision of **Asst. Prof. Ms. HITHA P S**. This submission represents my ideas in my own words and where ideas or words of others have been included, I have adequately and accurately cited and referenced the original sources. I also declare that I have adhered to ethics of academic honesty and integrity and have not misrepresented or fabricated any data or idea or fact or source in my submission. I understand that any violation of the above will be a cause for disciplinary action by the institute and/or the University and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been obtained. This report has not been previously formed the basis for the award of any degree, diploma or similar title of any other University.

KANJIRAPPALLY

TESSA JOSE

Date: 18-03-2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
AMAL JYOTHI COLLEGE OF ENGINEERING
KANJIRAPPALLY



CERTIFICATE

This is to certify that the report entitled **“HOSPITAL ASSET MANAGEMENT AND TRACKING PORTAL”** submitted by **TESSA JOSE (AJC23CS199)** to the APJ Abdul Kalam Technological University in partial fulfillment of the requirements for the award of the Degree of Bachelor of Technology in Computer Science and Engineering is a Bonafide record of the project work carried out by her under my guidance and supervision. This report in any form has not been submitted to any other University or Institute for any purpose.

Ms. Hitha P S

Internal Supervisor

Ms. Bini M Issac

Project Coordinator

Dr. Juby Mathew

Head of Department, CSE



Vision of the Department

The Computer Science and Engineering department is committed to continually improve the educational environment in order to develop professionals with strong technical and research backgrounds.

Mission of the Department

To provide quality education in both theoretical and applied foundations of Computer Science & Engineering.
Create highly skilled Computer Engineers, capable of doing research and also develop solutions for the betterment of the nation.
Inculcate professional and ethical values among students.
Support society by participating in and encouraging technology transfer.

Programme Educational Objectives

- PEO1** Be successfully employed in computing profession as well as multidisciplinary domains in supportive and leadership roles.
- PEO2** Participate in life-long learning through successful completion of advanced degrees, continuing education, certifications and/or other professional development.
- PEO3** Promote design, research, product implementation and services in the field of Computer Science and Engineering through strong technical, communication and entrepreneurial skills.

Programme Outcomes

B Tech Computer Science and Engineering Graduates will be able to:

1. Engineering knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
2. Problem analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
3. Design/development of solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
4. Conduct investigations of complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
5. Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modelling to complex engineering activities with an understanding of the limitations.
6. The engineer and society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
7. Environment and sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
8. Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
9. Individual and team work: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
10. Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
11. Project management and finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
12. Life-long learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

Programme Specific Outcomes

- PSO1** Apply Engineering knowledge to analyze, design and develop computing solutions by employing modern computer languages, environments and platforms that can solve complex problems.
- PSO2** Anticipate the changing direction of computational technology, evaluate it and communicate the likely utility of that for building software systems that would perform tasks related to industry, research and education.
- PSO3** Inculcate the knowledge of Engineering and Management principles to manage projects effectively and create innovative career paths.

ACKNOWLEDGEMENT

First of all, I sincerely thank the **Almighty GOD** who is most beneficent and merciful for giving me knowledge and courage to complete the Project successfully.

I derive immense pleasure in expressing my sincere thanks to our Manager and Managing Trustee, **Very Rev. Fr. Bobby Alex Mannamplackal**, **Rev. Fr. Dr. Roy Abraham Pazhayaparampil** (Director, Administration) and to our Principal, **Dr. Lillykutty Jacob** for the kind cooperation in all aspects of my Project.

I express my gratitude to **Dr. Juby Mathew**, HoD, Department of Computer Science and Engineering for his kind co-operation in all aspects of my Project. I express my sincere thanks to my internal guide, **Ms. Hitha P S**, Assistant Professor and my Project Coordinator, **Ms. Bini M Issac** Assistant Professor for their encouragement and motivation during the project.

I am indebted to my beloved teachers for their cooperation and suggestion throughout the project which helped me a lot. I also thank all my friends and classmates for their interest, dedication and encouragement shown towards the project. I convey hearty thanks to my parents for their moral support, suggestion and encouragement to make this venture a success.

ABSTRACT

Hospitals depend on a wide range of movable medical assets such as wheelchairs, stretchers, oxygen cylinders, and hospital beds to deliver effective healthcare services. Traditional manual methods of tracking these assets using registers or spreadsheets often lead to misplacement, delays, and inaccurate records, negatively affecting hospital workflow. The Hospital Asset Management and Tracking Portal addresses these issues by providing a centralized web-based platform built using Python and the Django framework, with SQLite as the database and HTML, CSS, and JavaScript for the user interface. The system enables administrators to register and maintain detailed asset records, while authorized staff can update asset locations, modify operational statuses, and track asset usage in real time. By digitizing the asset management process, the system improves resource visibility, reduces manual errors, and enhances operational efficiency, with the flexibility to migrate to enterprise-level databases such as MySQL for large-scale environments.

CONTENTS

CHAPTER 1: INTRODUCTION	1
1.1 Background of the Study.....	1
1.2 Problem Statement	1
1.3 Objectives	2
1.4 Scope of the Project	2
1.5 Organization of the Report.....	3
CHAPTER 2: LITERATURE REVIEW	4
2.1 Introduction to Existing Systems	4
2.2 Review of Related Works	4
2.2.1 RFID-Based Hospital Asset Tracking Systems	4
2.2.2 IoT-Enabled Medical Asset Management.....	5
2.2.3 ERP-Based Hospital Asset Modules	5
2.2.4 Spreadsheet-Based Inventory Management.....	5
2.2.5 Django-Based Web Applications for Inventory Management.....	5
2.3 Comparative Analysis	6
2.4 Limitations of Existing Systems	6
2.5 Research Gap Identification	7
CHAPTER 3: SOFTWARE REQUIREMENTS SPECIFICATION	8
3.1 Purpose.....	8
3.2 Overall Description	8
3.3 Functional Requirements.....	9
3.4 Non-Functional Requirements.....	10
3.4.1 Performance	10
3.4.2 Scalability.....	10
3.4.3 Security.....	11

3.4.4 Reliability	11
3.4.5 Usability	11
3.4.6 Maintainability	11
3.5 Constraints	11
3.6 Assumptions and Dependencies	12
CHAPTER 4: SYSTEM ANALYSIS AND DESIGN	13
4.1 Overall System Architecture	13
4.2 Model Description	13
4.3 Data Flow Diagram	14
4.3.1 Level 0 DFD	14
4.3.2 Level 1 DFD	14
4.4 Use Case Diagram	15
4.5 Sequence Description	16
4.5.1 Take Asset Sequence	16
4.5.2 Asset Registration Sequence	17
4.6 ER Diagram	17
4.7 Database Design	18
4.8 Algorithm / Pseudocode	19
4.8.1 Asset Registration with Bulk Quantity	19
CHAPTER 5: IMPLEMENTATION	26
5.1 Development Environment	26
5.2 Module-wise Implementation	26
5.2.1 Models	26
5.2.2 Views	27
5.2.3 Templates	27
5.2.4 URL Configuration	27
5.2.5 Role-Based Access Control	27
CHAPTER 6: TESTING	29
6.1 Testing Strategy	29
6.2 Test Case Table	29
6.3 Testing Results	31
CHAPTER 7: RESULTS AND DISCUSSION	32
7.1 Output Results	32
7.2 Comparison with Existing Systems	33

7.3 Observations	33
CHAPTER 8: CONCLUSION AND FUTURE WORKS.....	35
8.1 Conclusion	35
8.2 Future Scope	35
REFERENCES	37
APPENDIX.....	38
Appendix A: Sample Code	38
A.1 Asset Model (models.py)	38
A.2 Dashboard View (views.py)	38
A.3 Add Asset View (views.py).....	39
Appendix B: Sample Screenshots.....	40

LIST OF FIGURES

4.1: Data Flow Diagram.....	15
4.2: Use Case Diagram.....	16
4.3: ER Diagram.....	18
A.1: User Registration Page.....	41
A.2: User Login Page.....	41
A.3: Admin Dashboard Page.....	42
A.4: Admin View Asset Page.....	42
A.5: Admin Add Asset Page.....	43
A.6: Admin Asset Information Page.....	43
A.7: Admin Password Change Page.....	44
A.8: Staff Dashboard Page.....	44
A.9: Staff View Asset Page	45
A.10: Staff Asset Status Updation Page.....	45
A.11: Staff Password Change Page	46
A.12: Role Based Login Page.....	46

LIST OF TABLES

2.1: Comparative Analysis.....	6
4.1: Assets_Category.....	18
4.2: Assets_Asset.....	19
5.1: Development Tools.....	26
6.1: Test Cases For Hospital Asset Management and Tracking Portal.....	29
7.1: Comparison With Existing Systems.....	33

CHAPTER 1

INTRODUCTION

1.1 Background of the Study

Hospitals rely on a wide range of medical equipment and movable assets such as wheelchairs, stretchers, oxygen cylinders, trolleys, and hospital beds to provide efficient healthcare services. Proper management and tracking of these assets are essential to ensure that medical equipment is available when required and utilized effectively across different hospital departments. However, in many healthcare institutions, asset information is still maintained manually using paper records or spreadsheets. This traditional method often leads to equipment misplacement, delays in locating essential medical devices, and inaccurate record keeping.

With the advancement of information technology, digital systems have become essential for improving operational efficiency in healthcare environments. A centralized digital platform can help hospitals manage their assets more effectively by providing real-time access to asset information. Such systems enable administrators and staff to monitor equipment availability, update asset locations, and maintain accurate records of asset usage.

The Hospital Asset Management and Tracking Portal is designed to provide a centralized web-based platform for managing and monitoring hospital assets. The system enables authorized users to log into the portal and access asset information through an intuitive interface. By digitizing the asset management process, hospitals can improve resource visibility, reduce manual errors, and ensure better utilization of critical medical equipment.

1.2 Problem Statement

In many hospitals, the management and tracking of medical equipment are still performed manually using paper records or spreadsheets. These manual methods are inefficient and prone to errors, which often leads to equipment misplacement, inaccurate record maintenance, and delays in locating critical assets. When hospital staff are unable to quickly identify the location or status of

medical equipment, it can negatively impact the quality and efficiency of healthcare services.

The absence of a centralized asset management system also makes it difficult for hospital administrators to monitor asset usage and availability across different departments. Without proper tracking, hospitals may face issues such as underutilization of equipment, loss of assets, and inefficient resource allocation.

Therefore, there is a need for an efficient and reliable digital system that can manage and track hospital assets in a structured and organized manner. A web-based portal that allows asset registration, location tracking, and status monitoring can significantly improve hospital asset management and ensure better utilization of medical equipment.

1.3 Objectives

The primary objective of the Hospital Asset Management and Tracking Portal is to develop a web-based system that enables hospitals to efficiently manage and track movable medical assets.

The specific objectives of the system are as follows:

- To develop a centralized portal for storing and managing hospital asset information.
- To register and maintain detailed records of hospital equipment including asset name, category, serial number, location, and quantity.
- To allow authorized users to update asset location when equipment is moved between departments.
- To track the status of assets such as available, in use, or under maintenance.
- To maintain records of asset allocation and usage by hospital staff.
- To improve the efficiency, accuracy, and reliability of hospital asset management.

1.4 Scope of the Project

The Hospital Asset Management and Tracking Portal focuses on improving the management and monitoring of movable medical equipment within a hospital environment. The system provides functionality for registering hospital assets, updating asset details, managing asset status, and tracking asset movement between departments through a centralized digital platform.

Authorized hospital staff can log into the system to access asset information, update asset locations,

and monitor asset availability. The portal allows administrators to maintain accurate records of hospital equipment and ensure that critical medical assets are available when required.

The current implementation of the system uses SQLite as the database for storing and managing asset information efficiently. In the future, the system can be enhanced by integrating advanced technologies such as RFID or IoT-based asset tracking for real-time monitoring of equipment. Additionally, the system can be migrated to MySQL or other enterprise-level database systems to support large-scale hospital environments with multiple users and larger data volumes.

1.5 Organization of the Report

This report is organized into several chapters that describe the design, development, and implementation of the proposed system. Chapter 1 provides an introduction to the project, including the background of the study, problem statement, objectives, and scope of the system. Chapter 2 presents the literature review and discusses existing systems and related research in the area of hospital asset management and tracking. Chapter 3 describes the Software Requirements Specification (SRS), including functional and non-functional requirements of the proposed system. Chapter 4 explains the system analysis and design, including system architecture, module descriptions, and design diagrams such as data flow diagrams, use case diagrams, sequence diagrams, and entity relationship diagrams. Chapter 5 discusses the implementation details and technologies used in developing the system. Chapter 6 presents the testing strategy and test cases used to evaluate the system. Chapter 7 discusses the results obtained from the implemented system and observations from the project. Chapter 8 concludes the report and highlights possible future improvements and enhancements of the system.

CHAPTER 2

LITERATURE REVIEW

2.1 Introduction to Existing Systems

Hospitals use a large number of movable medical assets such as wheelchairs, stretchers, oxygen cylinders, trolleys, and hospital beds. Proper management of these assets is important to ensure that equipment is available when needed and used efficiently across hospital departments. In many hospitals, asset details are still recorded manually using paper registers or spreadsheets. These methods are time-consuming and often lead to errors, misplacement of equipment, and difficulty in locating medical assets during emergencies.

To improve hospital operations, many healthcare institutions have started adopting digital asset management systems. These systems store asset information in centralized databases and allow hospital staff to update and monitor equipment details through computer-based applications. Digital systems improve visibility of hospital resources and reduce manual errors in asset management.

2.2 Review of Related Works

2.2.1 RFID-Based Hospital Asset Tracking Systems

Several studies have explored the use of Radio Frequency Identification (RFID) technology for real-time tracking of medical equipment in hospitals. Systems proposed by researchers such as Ting et al. (2011) demonstrated that RFID based tracking significantly reduces the time nurses spend searching for equipment. However, these systems require dedicated hardware infrastructure including RFID readers and tags, which involves high deployment and maintenance costs. Such solutions are often economically unfeasible for smaller hospitals in developing countries. Furthermore, RFID systems demand ongoing technical expertise for calibration, tag replacement, and reader maintenance, placing an additional burden on hospital IT staff. Despite their accuracy, the scalability limitations and infrastructure dependency of RFID based solutions make them impractical as a universal standard for asset management across diverse healthcare settings.

2.2.2 IoT-Enabled Medical Asset Management

Studies in recent years have proposed Internet of Things (IoT) based asset management systems where medical devices are embedded with sensors that communicate their location and status to a central server. These systems provide granular real-time data but come with significant implementation complexity. The dependency on stable network connectivity and specialized sensor hardware limits their applicability in resource-constrained hospital environments.

2.2.3 ERP-Based Hospital Asset Modules

Enterprise Resource Planning (ERP) systems such as SAP and Oracle provide modules for asset management as part of broader hospital management solutions. While feature-rich, these platforms are expensive, complex to configure, and typically designed for large multi-specialty hospitals. Small and medium-sized hospitals find it difficult to justify the cost and training effort required for such systems.

2.2.4 Spreadsheet-Based Inventory Management

Many hospitals continue to use Microsoft Excel or Google Sheets for asset tracking. These tools are accessible and easy to use but lack multi-user concurrency, access control, and real-time status tracking. Version conflicts, accidental deletion, and lack of audit trails are common issues that reduce the reliability of spreadsheet-based systems.

2.2.5 Django-Based Web Applications for Inventory Management

Django has been widely used in academic and small-scale commercial projects for building inventory and asset tracking systems due to its rapid development capabilities and built-in security features. Studies have demonstrated that Django's ORM, authentication framework, and admin interface significantly reduce development time while ensuring code quality. These characteristics make Django well-suited for developing the proposed hospital asset management portal. Moreover, Django's active open-source community and extensive documentation provide reliable support for iterative development and long-term maintainability, further reinforcing its suitability as the core framework for this system.

2.3 Comparative Analysis

Table 2.1 presents a comparison of existing asset management approaches with the proposed system.

Table 2.1 : Comparative Analysis

System / Approach	Technology	Cost	Real-Time Tracking	Role-Based Access	Scalability
RFID-Based Tracking	RFID Hardware	High	Yes	Limited	Moderate
IoT-Based Systems	IoT Sensors	Very High	Yes	Limited	Moderate
ERP Modules (SAP/Oracle)	Enterprise ERP	Very High	Yes	Yes	High
Spreadsheet-Based	MS Excel / Google Sheets	Low	No	No	Low
Proposed System	Django, SQLite, HTML/CSS	Low	Yes	Yes	High

2.4 Limitations of Existing Systems

Based on the review of related works, several limitations were identified in existing hospital asset management approaches. RFID and IoT-based systems require specialized hardware, making them expensive and difficult to deploy in resource-limited settings. ERP-based solutions are feature-rich but overly complex and costly for small and medium hospitals. Spreadsheet-based methods lack real-time tracking, multi-user access control, and audit capabilities. Most existing software solutions are either too general-purpose or too complex, lacking the simplicity and accessibility needed by hospital staff with limited technical expertise. This gap highlights the need for a lightweight, web-based system that balances functional completeness with ease of use, requiring no specialised hardware and minimal technical overhead for deployment and day-to-day operation.

2.5 Research Gap Identification

From the analysis of existing systems, it is clear that many asset management solutions either require expensive technologies such as RFID and IoT or involve complex enterprise platforms that are difficult to implement. Simpler methods such as spreadsheets lack essential features like centralized data management, real-time tracking, and controlled user access. Therefore, there is a need for a simple and cost-effective web-based system that allows hospitals to manage asset records, update equipment locations, and monitor asset status through an easy-to-use platform.

CHAPTER 3

SOFTWARE REQUIREMENTS SPECIFICATION

3.1 Purpose

The Software Requirements Specification (SRS) document describes the functional and non-functional requirements of the Hospital Asset Management and Tracking Portal. The document outlines the features, system behavior, and operational environment required for the system to function effectively. It serves as a reference for system development and helps ensure that the system meets the needs of hospital administrators and staff responsible for managing medical equipment.

3.2 Overall Description

3.2.1 Product Perspective

The Hospital Asset Management and Tracking Portal is a standalone web-based application. It interacts with a SQLite database for persistent storage of asset records and user data. The system operates within a standard web browser environment and requires no additional hardware or client-side software installations. The application is built on the Django framework and communicates with the database through Django's Object-Relational Mapper (ORM).

3.2.2 Product Functions

The major functions of the system include:

- User authentication and login for authorized staff
- Registration and management of hospital assets
- Categorization of assets and storage of asset details
- Updating asset location when equipment is moved between departments
- Monitoring asset status such as available, in use, or under maintenance
- Maintaining records of asset allocation and usage
- Viewing and searching asset information

3.2.3 User Classes

The system includes the following user roles:

1. Administrator - Responsible for managing asset records, adding new assets, updating asset details, and monitoring asset usage across hospital departments.
2. Authorized Staff - Hospital staff members who can view asset details and update asset status or location when equipment is used or transferred.

3.2.4 Operating Environment

The Hospital Asset Management and Tracking Portal operates in a web-based environment. The system is developed using Python with the Django framework for backend development. The user interface is designed using HTML, CSS, and JavaScript. SQLite is used as the database for storing asset records. The system can be accessed through common web browsers such as Google Chrome or Mozilla Firefox. The development of the application is carried out using Visual Studio Code on a Windows operating system.

3.3 Functional Requirements

The following functional requirements define the expected behavior of the system:

FR1 – User Authentication: The system shall provide a secure login interface for all users. Unauthorized access to any page shall redirect the user to the login page.

FR2 – Role-Based Access Control: The system shall distinguish between Administrator and Staff roles. Administrators shall have full CRUD (Create, Read, Update, Delete) access to assets. Staff shall be restricted to viewing, taking, returning, and updating location.

FR3 – Asset Registration: Administrators shall be able to register new assets with the following details: asset name, category, serial number, purchase date, location, quantity, price, and status.

FR4 – Multiple Quantity Registration: When an asset is registered with a quantity greater than one, the system shall automatically create individual records for each unit with unique serial numbers suffixed numerically.

FR5 – Asset Listing and Search: The system shall display a list of all registered assets with their name, serial number, status, location, and price. A search feature shall allow filtering by name,

serial number, or location.

FR6 – Asset Status Management: Authorized users shall be able to update the status of an asset to Available, In Use, or Under Maintenance.

FR7 – Asset Location Update: Authorized users shall be able to update the current location of an asset when it is moved to a different department or ward.

FR8 – Take Asset: Staff members shall be able to mark an available asset as In Use, which records the staff member's identity and the time of allocation.

FR9 – Return Asset: Staff members shall be able to return an asset that is currently In Use, resetting its status to Available and clearing the usage record.

FR10 – Asset Deletion: Administrators shall be able to delete an asset record from the system after a confirmation prompt.

FR11 – Asset Information View: Administrators shall be able to view detailed information for each asset including category, usage history, purchase details, and last location update timestamp.

FR12 – Dashboard: The system shall display a dashboard showing total asset count and counts broken down by status (Available, In Use, Under Maintenance), along with a summary table of asset availability.

FR13 – Password Change: Authenticated users shall be able to change their account password through a dedicated form.

3.4 Non-Functional Requirements

3.4.1 Performance

The system shall load all pages within three seconds under normal network conditions. Database queries shall be optimized using Django's ORM to minimize response time. The dashboard shall aggregate asset statistics efficiently using database-level annotation queries.

3.4.2 Scalability

The system is designed to support migration from SQLite to MySQL or PostgreSQL to accommodate larger datasets and multiple concurrent users. The Django framework's database abstraction layer ensures that such migration can be performed with minimal code changes.

3.4.3 Security

All pages shall be protected by Django's login_required decorator to prevent unauthorized access. CSRF protection shall be enforced on all forms using Django's built-in CSRF middleware. Passwords shall be stored as salted hashes using Django's authentication framework. Role-based access shall be enforced at the view level to prevent privilege escalation.

3.4.4 Reliability

The system shall maintain data integrity through Django's ORM transaction management. Deletion of assets shall be preceded by a confirmation prompt to prevent accidental data loss. The system shall provide informative error and success messages after every operation.

3.4.5 Usability

The interface shall be clean, intuitive, and responsive, designed using Bootstrap 5 to ensure compatibility across devices including desktops, tablets, and mobile phones. Color-coded status badges shall provide at-a-glance information about asset availability. Navigation shall be consistent across all pages.

3.4.6 Maintainability

The system shall follow Django's MVT (Model-View-Template) architectural pattern to ensure clean separation of concerns. Code shall be modular and well-documented to facilitate future maintenance and feature additions.

3.5 Constraints

The system currently operates with SQLite as the database, which limits concurrent write performance. The system does not include barcode or QR code scanning functionality. Internet connectivity is required for accessing Bootstrap and Bootstrap Icons CDN resources in the current implementation. The system is designed for deployment within a local area network (LAN) environment. The current implementation does not support automated notifications or alerts, meaning administrators must manually monitor asset statuses and availability without any system-driven reminders. Additionally, the system lacks a dedicated mobile application, restricting access to desktop and laptop browsers and limiting usability for staff who rely primarily on smartphones or tablets in a clinical setting.

3.6 Assumptions and Dependencies

It is assumed that all users have basic computer literacy and are familiar with web browser navigation. The system assumes that user accounts are pre-created by the Administrator through Django's admin interface. The application depends on Python 3.x, Django 4.x or later, and Bootstrap 5 for its operation.

CHAPTER 4

SYSTEM ANALYSIS AND DESIGN

4.1 Overall System Architecture

The Hospital Asset Management and Tracking Portal follows a web-based client-server architecture. In this architecture, users interact with the system through a web browser, which acts as the client interface. The application logic is handled by the Django framework on the server side, which processes user requests and communicates with the database. Asset information and user data are stored in the SQLite database. When users perform actions such as adding assets or updating asset status, the request is processed by the server and the updated information is stored in the database. This architecture allows centralized data management and ensures that authorized users can access the system through a secure web interface.

4.2 Model Description

The system is organized into the following modules:

Authentication Module: Handles user login, logout, and password change functionality using Django's built-in authentication framework. All application pages are secured using the `login_required` decorator.

Dashboard Module: Displays an overview of the hospital asset inventory, including total asset count and counts broken down by status. Provides an asset availability summary table for Staff users.

Asset Registration Module: Allows Administrators to add new assets to the system. Supports bulk registration by accepting a quantity value and automatically generating individual asset records with unique serial numbers.

Asset Listing and Search Module: Displays all registered assets in a tabular format. Provides a search bar to filter assets by name, serial number, or location. Shows role-appropriate action buttons for each asset.

Asset Management Module: Enables updating of asset status and location. Handles Take and Return operations for Staff users, recording the user identity and timestamp of each allocation.

Asset Information Module: Provides a detailed view of a specific asset including usage history, purchase details, and last location update time. Accessible to Administrators only.

Asset Deletion Module: Allows Administrators to delete asset records after a browser-level confirmation prompt.

4.3 Data Flow Diagram

4.3.1 Level 0 DFD (Context Diagram)

At the context level, the system has two external entities: Administrator and Staff. The Administrator interacts with the system to manage assets (add, update, delete) and view detailed information. The Staff interacts with the system to view assets, take available equipment, return equipment, and update locations. The system processes these inputs and stores and retrieves data from the Asset Database.

4.3.2 Level 1 DFD

At Level 1, the system is decomposed into the following processes: User Authentication, which validates login credentials and manages sessions; Asset Registration, which receives asset data from the Administrator and stores it in the database; Asset Listing, which retrieves asset records from the database and presents them to users; Status and Location Update, which processes update requests from both Administrators and Staff; Take/Return Asset, which updates the allocation status, records the user and timestamp, or clears the allocation on return; and Dashboard Aggregation, which queries the database for status-wise asset counts and presents them on the dashboard.

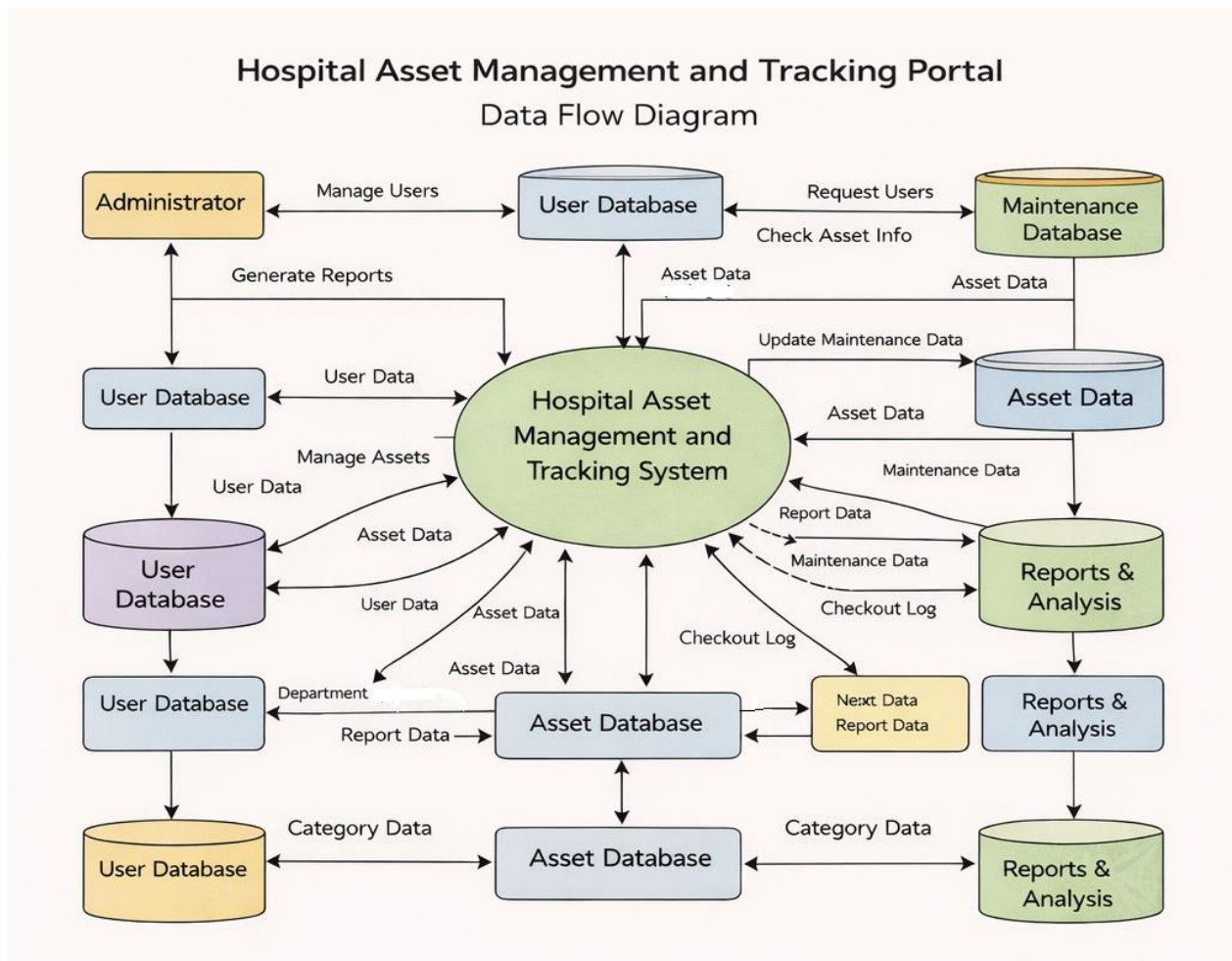


Figure 4.1 : Data Flow Diagram

4.4 Use Case Diagram

The system has two primary actors: Administrator and Staff. The Administrator can perform the following use cases: Login, View Dashboard, Add Asset, View Asset List, Search Asset, Update Asset Status, Update Asset Location, View Asset Information, Delete Asset, and Change Password. The Staff can perform: Login, View Dashboard, View Asset List, Search Asset, Take Asset, Return Asset, Update Asset Location, Update Asset Status, and Change Password. While both actors share common functionalities such as login, dashboard access, and asset search, the Administrator holds elevated privileges that encompass the full asset lifecycle management, whereas Staff interactions are intentionally scoped to operational tasks that reflect their day-to-day responsibilities within the hospital environment.

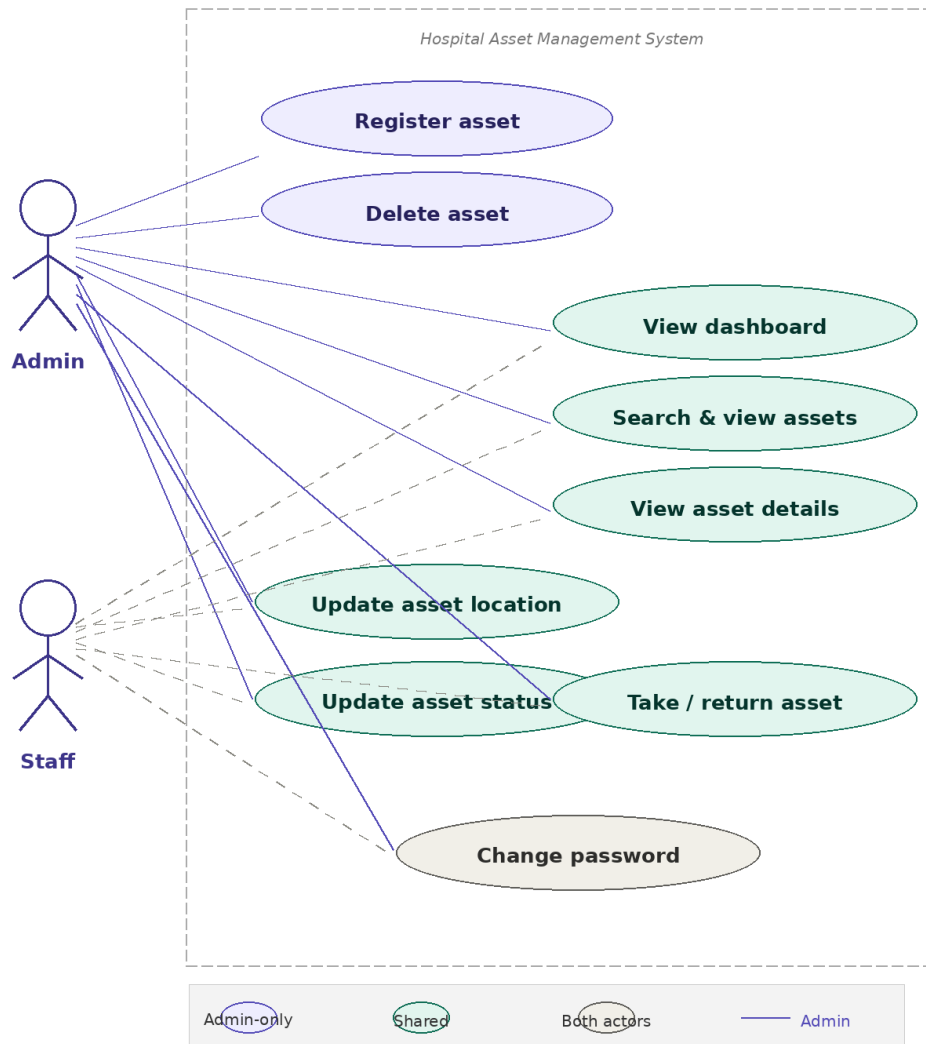


Figure 4.2 : Use Case Diagram

4.5 Sequence Description

4.5.1 Take Asset Sequence

The sequence for the Take Asset operation is as follows: The Staff user navigates to the Asset List page and clicks the Take button for an available asset. The browser sends an HTTP GET request to the take_asset URL with the asset primary key. The Django view retrieves the asset from the database and checks that its status is Available. If available, the view updates the status to In Use, sets the taken_by field to the current user, and records the current timestamp in the taken_at field. The changes are saved to the database, a success message is generated, and the user is redirected to the Asset List page. If the asset is found to be unavailable at the time of the request, the view rejects the operation and returns an appropriate error message, preventing concurrent allocation

conflicts and maintaining data integrity. The entire sequence is protected by the `login_required` decorator, ensuring that only authenticated Staff members can initiate a Take Asset operation, thereby enforcing role-based access control at the view level.

4.5.2 Asset Registration Sequence

The Administrator navigates to the Add Asset page. The system renders the asset registration form. The Administrator fills in the form and submits it. The view validates the form data. If valid, the Asset model's save method checks whether the quantity is greater than one. If so, the first unit is saved with the base serial number, and subsequent units are created with serial numbers suffixed as base-2, base-3, and so on. A success message is shown and the Administrator is redirected to the Asset List. If the form validation fails due to missing or incorrect field entries, the system re-renders the registration form with inline error messages highlighting the specific fields that require correction, allowing the Administrator to amend and resubmit without losing previously entered data. Each asset record created through this sequence is automatically associated with the currently authenticated Administrator via the `added_by` field, establishing a clear audit trail that tracks which user registered each asset and when. In cases where a single unit is being registered, the bulk creation logic is bypassed entirely and the asset is saved directly with the exact serial number provided, ensuring that the registration sequence remains efficient and avoids unnecessary processing overhead for standard single-unit entries.

4.6 ER Diagram

The database consists of the following entities and their relationships. The User entity (provided by Django's built-in authentication system) has attributes: `id`, `username`, `password`, `email`, `first_name`, `last_name`, and `group membership`. The Category entity has attributes: `id` and `name` and `description`. The Asset entity has attributes: `id`, `name`, `serial_number`, `purchase_date`, `location`, `status`, `price`, `quantity`, `taken_at`, `location_updated_at`, and foreign keys to Category, `added_by` (User), and `taken_by` (User). The relationships are: Category has a one-to-many relationship with Asset (one category can have many assets). User has a one-to-many relationship with Asset through the `added_by` field. User has a one-to-many relationship with Asset through the `taken_by` field (representing current allocation).

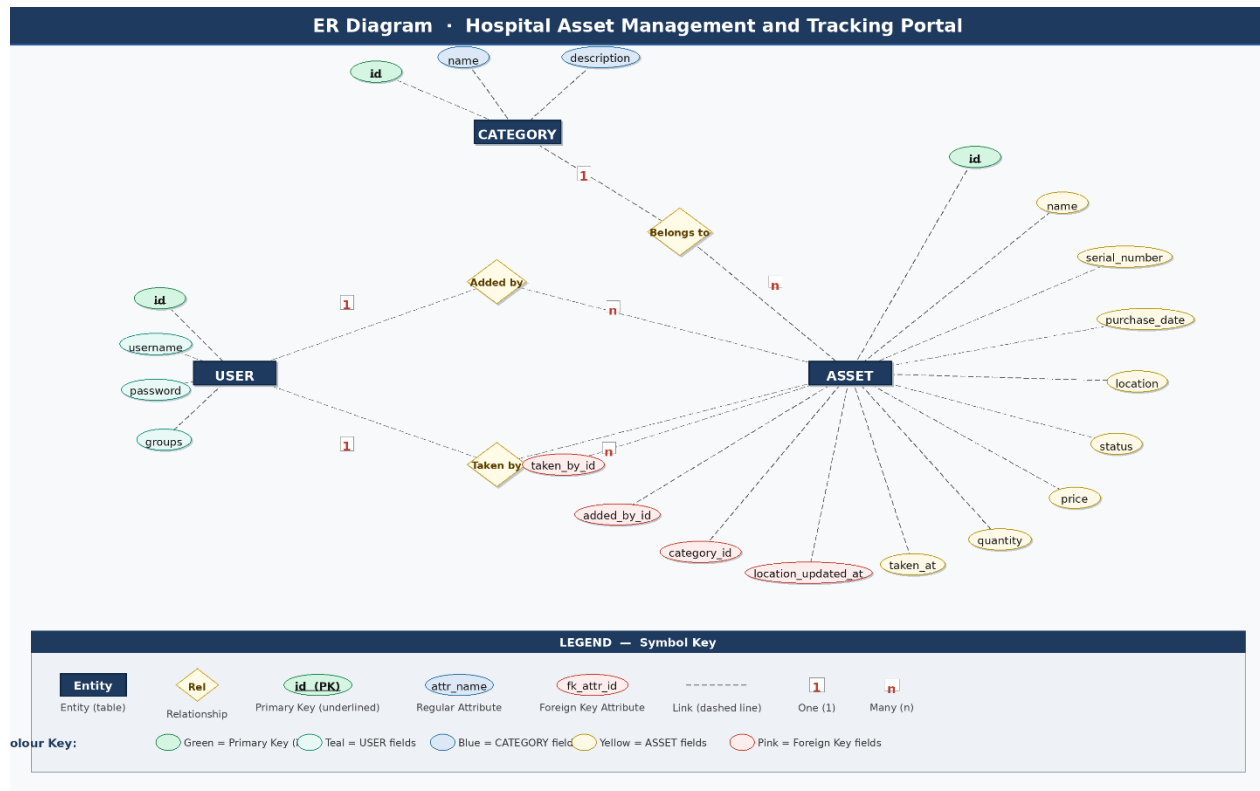


Figure 4.3 : ER Diagram

4.7 Database Design

The system uses two primary database tables in addition to Django's built-in authentication tables.

Table 4.1: Assets_Category

Field Name	Data Type	Constraints	Description
id	INTEGER	Primary Key, Auto	Unique category identifier
name	VARCHAR(100)	NOT NULL	Name of the asset category
description	TEXT	NULLABLE	Optional description of the category

Table 4.2: Assets_Asset

Field Name	Data Type	Constraints	Description
id	INTEGER	Primary Key, Auto	Unique asset identifier
name	VARCHAR(200)	NOT NULL	Name of the asset
category_id	INTEGER	Foreign Key → Category	Asset category reference
serial_number	VARCHAR(100)	UNIQUE, NOT NULL	Unique serial number
purchase_date	DATE	NOT NULL	Date of asset purchase
location	VARCHAR(200)	NOT NULL	Current location/department
status	VARCHAR(50)	NOT NULL	available / in_use / maintenance
price	DECIMAL(10,2)	Default 0.00	Purchase price in INR
quantity	INTEGER	Default 1	Quantity (used during bulk add)
added_by_id	INTEGER	FK → User, NULLABLE	User who registered the asset
taken_by_id	INTEGER	FK → User, NULLABLE	User who currently holds the asset
taken_at	DATETIME	NULLABLE	Timestamp when asset was taken
location_updated_at	DATETIME	Auto, NOT NULL	Last location update timestamp

4.8 Algorithm / Pseudocode

4.8.1 Asset Registration with Bulk Quantity

ALGORITHM: BulkAssetRegistration.

INPUT: AssetForm data (name, category, serial_number, purchase_date, location, status, price, quantity).

OUTPUT: Individual asset records created in database.

BEGIN

```
1. Validate submitted form data
2. IF form is valid THEN
    3. Set asset.added_by = current logged-in user
    4. Store quantity = asset.quantity
    5. IF quantity > 1 THEN
        6. Save first asset with base serial number, quantity = 1
        7. FOR i = 2 TO quantity DO
            8. Create new asset with serial_number = base_serial + '-' + i
            9. quantity = 1 for each created record
        10. END FOR
    11. ELSE
        12. Save asset normally
    13. END IF
    14. Show success message
    15. Redirect to Asset List
16. ELSE
    17. Re-render form with validation errors
18. END IF
END
```

4.8.2 User Login

ALGORITHM: UserLogin

INPUT: username, password

OUTPUT: Authenticated session or error message

BEGIN

1. User submits login form with username and password
2. Django authenticates credentials against the User table
3. IF authentication succeeds THEN
 4. Create user session
 5. Determine user role (Admin or Staff) via Groups
 6. Redirect to dashboard
7. ELSE
 8. Display error: Invalid username or password

9. Reload login form

10. END IF

END

4.8.3 Dashboard Aggregation

ALGORITHM: DashboardAggregation

INPUT: Authenticated user session

OUTPUT: Asset counts by status, summary table

BEGIN

1. Verify user is authenticated (login_required)
2. Query Asset table:
 3. total_assets = COUNT(all assets)
 4. available = COUNT(assets WHERE status = 'available')
 5. in_use = COUNT(assets WHERE status = 'in_use')
 6. maintenance = COUNT(assets WHERE status = 'maintenance')
7. GROUP assets BY name, annotate with per-status counts
8. Check if user belongs to Admin or Staff group
9. Render dashboard template with all counts and summary

END

4.8.4 Asset Listing and Search

ALGORITHM: AssetListingAndSearch

INPUT: Optional search query string q

OUTPUT: Filtered list of assets

BEGIN

1. Retrieve all Asset records from the database
2. IF search query q is provided THEN
 3. Filter assets WHERE:
 - name CONTAINS q
 - OR serial_number CONTAINS q
 - OR location CONTAINS q
4. END IF
5. Compute summary: GROUP BY name, COUNT by status
6. Determine user role (Admin / Staff)

7. Render asset list template with assets, summary, and role flags

END

4.8.5 Take Asset

ALGORITHM: TakeAsset

INPUT: asset primary key (pk), authenticated user

OUTPUT: Asset status updated to 'in_use', usage recorded

BEGIN

1. Retrieve asset by pk; raise 404 if not found
2. IF asset.status == 'available' THEN
 3. Set asset.status = 'in_use'
 4. Set asset.taken_by = current_user
 5. Set asset.taken_at = current_timestamp
 6. Save asset to database
 7. Display success message
8. ELSE
 9. Display error: Asset is not available
10. END IF
11. Redirect to asset list

END

4.8.6 Return Asset

ALGORITHM: ReturnAsset

INPUT: asset primary key (pk), authenticated user

OUTPUT: Asset status reset to 'available', usage record cleared

BEGIN

1. Retrieve asset by pk; raise 404 if not found
2. IF asset.status == 'in_use' THEN
 3. Set asset.status = 'available'
 4. Set asset.taken_by = NULL
 5. Set asset.taken_at = NULL
 6. Save asset to database
 7. Display success message
8. ELSE

9. Display error: Asset is not currently in use
10. END IF
11. Redirect to asset list

END

4.8.7 Update Asset Location

ALGORITHM: UpdateAssetLocation

INPUT: asset primary key (pk), new location string

OUTPUT: Asset location updated, timestamp refreshed

BEGIN

1. Retrieve asset by pk; raise 404 if not found
2. IF request method is POST THEN
 3. Bind submitted data to UpdateLocationForm
 4. IF form is valid THEN
 5. Save updated location to asset record
 6. location_updated_at is auto-updated by Django ORM
 7. Display success message
 8. Redirect to asset list
 9. ELSE
 10. Re-render form with validation errors
11. END IF
12. ELSE
 13. Render empty UpdateLocationForm for the asset
14. END IF

END

4.8.8 Update Asset Status

ALGORITHM: UpdateAssetStatus

INPUT: asset primary key (pk), new status value

OUTPUT: Asset status updated in database

BEGIN

1. Retrieve asset by pk; raise 404 if not found
2. IF request method is POST THEN
 3. Bind submitted data to UpdateStatusForm

```
4. IF form is valid THEN
    5. Save updated status to asset record
    6. Display success message
    7. Redirect to asset list
8. ELSE
    9. Re-render form with validation errors
10. END IF
11. ELSE
    12. Render empty UpdateStatusForm for the asset
13. END IF
END
```

4.8.9 Delete Asset

ALGORITHM: DeleteAsset

INPUT: asset primary key (pk), authenticated Admin user

OUTPUT: Asset record removed from database

BEGIN

```
1. Check if current user belongs to Admin group
2. IF user is NOT Admin THEN
    3. Display error: Only admins can delete assets
    4. Redirect to asset list
5. END IF
6. Retrieve asset by pk; raise 404 if not found
7. User confirms deletion via browser confirmation dialog
8. IF user confirms THEN
    9. Delete asset record from database
    10. Display success message
    11. Redirect to asset list
12. ELSE
    13. No action taken; asset remains in database
14. END IF
```

END

4.8.10 View Asset Information

ALGORITHM: ViewAssetInformation

INPUT: asset primary key (pk), authenticated Admin user

OUTPUT: Full asset details rendered on screen

BEGIN

1. Verify user is authenticated (login_required)
2. Retrieve asset by pk; raise 404 if not found
3. Pass asset object to asset_info template
4. Template renders the following fields:
 - name, serial_number, status, location, category
 - purchase_date, price, added_by, location_updated_at
5. IF asset.status == 'in_use' THEN
 6. Also render: taken_by (username), taken_at (timestamp)
7. END IF

END

CHAPTER 5

IMPLEMENTATION

5.1 Development Environment

The Hospital Asset Management and Tracking Portal was developed using the following tools and technologies:

Table 5.1: Development Tools

Component	Technology / Tool	Version / Details
Programming Language	Python	3.13
Web Framework	Django	4.x
Database	SQLite	Bundled with Python
Frontend Framework	Bootstrap	5.3 (CDN)
Icon Library	Bootstrap Icons	1.x (CDN)
Template Engine	Django Templating Language (DTL)	Built-in
IDE / Editor	Visual Studio Code	Latest
Version Control	Git	Local
Operating System	Windows 10/11	-

5.2 Module-wise Implementation

5.2.1 Models

Two Django model classes were defined: Category and Asset. The Category model stores the asset category name and description. The Asset model contains fields for name, category (ForeignKey), serial number, purchase date, location, status (choices: available, in_use, maintenance), price, quantity, added_by (ForeignKey to User), taken_by (ForeignKey to User), taken_at (DateTimeField), and location_updated_at (auto-updated DateTimeField). The Asset model overrides the save() method to implement bulk creation: when a new asset is saved with quantity

greater than one, it creates individual Asset records with unique serial numbers suffixed as base-2, base-3, and so on.

5.2.2 Views

All views are protected with the `@login_required` decorator. The dashboard view aggregates asset counts using Django's `annotate()` and `Count()` functions with conditional filters for each status. The `asset_list` view supports search functionality by filtering assets using Q objects across name, serial_number, and location fields. The `asset_registration` view checks that only users in the Admin group can add assets. The `take_asset` view verifies asset availability before marking it as In Use and recording the user and timestamp. The `return_asset` view resets the status and clears the `taken_by` and `taken_at` fields. The `update_location` and `update_status` views use model forms with restricted fields to allow partial updates.

5.2.3 Templates

Templates are organized in the `templates/assets/` directory. A base template (`base.html`) provides the common navigation bar, Bootstrap CSS/JS imports, and a content block that child templates extend. The dashboard template (`dashboard.html`) renders four summary cards for total, available, in-use, and maintenance asset counts, and conditionally shows the asset availability summary table for Staff users. The `asset_list` template (`asset_list.html`) renders the summary table, a search form, and a detailed asset table with role-appropriate action buttons. The `asset_info` template (`asset_info.html`) shows all fields of a specific asset, including usage information when the asset is in use.

5.2.4 URL Configuration

URL routing is configured in `assets/urls.py`. The routes defined include the dashboard at the root URL, asset list, asset registration, asset detail (info), take, return, update location, update status, and delete operations. The `hospitalassetproject/urls.py` includes the assets app URLs under the root path and Django's built-in authentication URLs (login, logout, password change) under the registration path.

5.2.5 Role-Based Access Control

Role-based access control is implemented using Django's built-in Groups framework, with two distinct groups defined: **Admin** and **Staff**. Each view function enforces access restrictions by verifying group membership through `request.user.groups.filter(name='Admin').exists()`, ensuring

that only authorised users can execute sensitive operations. Administrators have unrestricted access to all system functions, including asset registration, modification, and deletion, while Staff members are limited to taking assets, returning them, and updating asset locations. At the presentation layer, Django Template Language (DTL) template tags evaluate the authenticated user's group membership at runtime, dynamically rendering the appropriate action buttons, status badges, and navigation options to deliver a role-appropriate interface without exposing unauthorised controls.

CHAPTER 6

TESTING

6.1 Testing Strategy

Testing is an important phase in software development that ensures the system functions correctly and satisfies the specified requirements. The Hospital Asset Management and Tracking Portal was tested to verify that all modules operate properly and that the system provides accurate results when users perform different operations.

The testing process mainly focused on functional testing, where individual features of the system were tested to ensure that they behave as expected. The system was tested using different input scenarios to verify that the application correctly processes asset data, updates asset records, and retrieves information from the database.

Both user-side operations and administrator-side operations were tested during the testing process. User functionalities such as logging into the system, viewing asset information, searching for assets, and updating asset locations were verified. Administrative functions such as adding new assets, updating asset details, managing categories, and viewing asset records were also tested.

The interaction between the web interface, backend server, and database was evaluated to ensure smooth communication between system components. Testing also verified that the system correctly stores and retrieves asset information without data inconsistency.

6.2 Test Case Table

Table 6.1: Test Cases for Hospital Asset Management and Tracking Portal

TC ID	Test Case Description	Input	Expected Output	Actual Output	Status
TC01	Login with valid credentials	Valid username and password	Redirect to Dashboard	Redirected to Dashboard	Pass

TC02	Login with invalid credentials	Wrong password	Error message displayed	Error message shown	Pass
TC03	Access asset list without login	Direct URL access	Redirect to Login page	Redirected to Login	Pass
TC04	Admin adds new single asset	Valid asset form data, quantity=1	Asset created, redirect to list	Asset created successfully	Pass
TC05	Admin adds asset with quantity=3	Valid data, quantity=3	3 asset records created with suffixed serial numbers	3 records created correctly	Pass
TC06	Add asset with duplicate serial number	Existing serial number	Form error shown	Validation error displayed	Pass
TC07	Staff user tries to add asset	Staff login, navigate to add URL	Access denied message	Redirected with error message	Pass
TC08	Search asset by name	Search query: 'Wheelchair'	Filtered list of matching assets	Correct filtered results shown	Pass
TC09	Search asset by serial number	Search query: 'WC-001'	Matching asset displayed	Correct asset shown	Pass
TC10	Staff takes available asset	Click Take on available asset	Status changes to In Use, user and time recorded	Status updated, record saved	Pass
TC11	Staff tries to take In Use asset	Click Take on in-use asset	Error message shown	Error message displayed	Pass
TC12	Staff returns In Use asset	Click Return on in-use asset	Status resets to Available, usage cleared	Status reset correctly	Pass
TC13	Update asset location	New location: ICU	Location updated, timestamp refreshed	Location updated successfully	Pass

TC14	Admin updates asset status to Maintenance	Select Maintenance in status form	Status updated to Maintenance	Status updated correctly	Pass
TC15	Admin views asset info page	Click Info on any asset	Detailed asset info displayed	All details shown correctly	Pass
TC16	Admin deletes asset with confirmation	Click Delete, confirm	Asset removed from list	Asset deleted successfully	Pass
TC17	Admin deletes asset, cancels confirmation	Click Delete, cancel	Asset remains in list	No deletion occurred	Pass
TC18	Dashboard displays correct counts	Multiple assets with different statuses	Correct counts shown for all statuses	Counts match database values	Pass
TC19	Change password with valid inputs	Current and new passwords	Password updated, session maintained	Password changed successfully	Pass
TC20	Staff user logout	Click Logout	Session ended, redirect to Login	Logout successful	Pass

6.3 Testing Results

The testing results indicate that the system successfully performs all major operations related to hospital asset management and tracking. The application correctly handles user authentication, asset registration, asset information updates, and asset tracking operations. The system also successfully retrieves asset records from the database and displays them through the web interface. During testing, all modules operated as expected without major errors. The system demonstrated stable performance when processing user requests and updating asset information in the database. These results confirm that the Hospital Asset Management and Tracking Portal meets the functional requirements specified during the system design phase.

CHAPTER 7

RESULTS AND DISCUSSION

7.1 Output Results

The Hospital Asset Management and Tracking Portal was successfully developed and tested. The system provides a fully functional web-based interface for managing hospital assets. The following key outputs were observed during testing and demonstration.

The Login Page provides a clean authentication interface. Users are redirected to the Dashboard upon successful login. Invalid credentials result in an appropriate error message without exposing any system details.

The Dashboard displays four summary cards showing Total Assets, Available, In Use, and Under Maintenance counts in real time. Staff users additionally see an Asset Availability Summary table listing all asset types along with their total, available, in-use, and maintenance counts. The dashboard also provides Quick Action buttons for navigation.

The Asset List page displays all registered assets in a structured table with columns for name, serial number, status, location, price, and actions. Status values are highlighted using color-coded badges: green for Available, yellow for In Use, and red for Maintenance. Role-appropriate action buttons are displayed for each asset. The search bar allows filtering by name, serial number, or location.

The Asset Registration form allows Administrators to enter all required asset details including name, category, serial number, purchase date, location, status, price, and quantity. When a quantity greater than one is entered, the system automatically creates multiple individual asset records with unique serial numbers, significantly reducing manual data entry effort.

The Asset Information page displays all details of a selected asset including category, purchase date, price, current location, added-by user, and last location update timestamp. When an asset is In Use, the page additionally shows the name of the staff member holding the asset and the exact time it was taken.

7.2 Comparison with Existing Systems

Table 7.1 below presents a comparison of the proposed system with existing hospital asset management approaches evaluated in the literature review.

Table 7.1: Comparison with Existing Systems

Feature	Proposed System	RFID-Based	ERP System	Spreadsheet
Cost	Low (Open Source)	High (Hardware)	Very High	Low
Real-Time Status	Yes	Yes	Yes	No
Role-Based Access	Yes	Limited	Yes	No
Web-Based Access	Yes	Depends	Yes	Limited
Usage History	Yes	Yes	Yes	No
Bulk Asset Add	Yes	No	Yes	Manual
Search Functionality	Yes	Yes	Yes	Limited
Hardware Required	No	Yes (RFID)	No	No
Scalability	High (MySQL ready)	Moderate	High	Low
Ease of Use	High	Moderate	Low	High

7.3 Observations

The system successfully fulfills all functional requirements specified in Chapter 3. All twenty test cases passed without errors. The role-based access control correctly restricted Staff users from adding or deleting assets while permitting all asset-viewing and allocation operations. The bulk asset registration feature functioned as designed, automatically creating individual records with unique serial numbers for multi-unit registrations.

The dashboard accurately reflected real-time asset counts and updated immediately after any status change. The search functionality returned correct filtered results across all three search fields. The take and return operations correctly recorded and cleared usage information, providing a reliable audit trail for asset allocation.

The user interface was well-received during demonstration, with stakeholders noting that the color-coded status badges and role-based button visibility made the system intuitive and easy to navigate without requiring technical training.

CHAPTER 8

CONCLUSION AND FUTURE WORKS

8.1 Conclusion

The Hospital Asset Management and Tracking Portal was successfully designed, developed, and tested as a mini project for the partial fulfillment of the Bachelor of Technology degree in Computer Science and Engineering. The system provides a lightweight, web-based solution for managing and tracking movable medical assets in hospital environments.

The proposed system addresses the key limitations of existing approaches by replacing manual, error-prone asset management processes with a centralized digital platform. The use of Django's MVT architecture, built-in authentication, ORM, and template engine enabled rapid and structured development. Role-based access control ensures that Administrators and Staff have appropriate levels of access, maintaining data security and integrity.

The system successfully demonstrates features including secure login, real-time dashboard statistics, asset registration with bulk quantity support, asset status and location management, take and return operations with usage tracking, asset detail views, and search functionality. All twenty test cases defined for the system passed successfully, confirming that the implementation meets the specified requirements.

By digitizing the asset management process, the system improves resource visibility, reduces manual errors, eliminates misplacement of critical equipment, and enhances the overall operational efficiency of hospital departments. The system is designed to be scalable and can be deployed in a production environment with minimal configuration changes.

8.2 Future Scope

The Hospital Asset Management and Tracking Portal can be extended in several directions to further enhance its functionality and applicability in real-world hospital environments. One key enhancement is QR code and barcode integration, where each registered asset is assigned a unique code that staff can scan using mobile devices to instantly retrieve asset information or perform take

and return operations. The system can also be extended to support preventive maintenance scheduling, enabling automated reminders to be sent to maintenance staff when routine servicing is due, thereby reducing equipment downtime. A comprehensive asset audit log can be implemented to record every change made to asset records, capturing details such as who made the change and when, which supports accountability and regulatory compliance. To improve operational convenience, a companion mobile application can be developed to allow hospital staff to manage assets directly from their smartphones. On the database side, the system can be migrated from SQLite to a production-grade relational database such as MySQL or PostgreSQL to accommodate larger datasets and higher volumes of concurrent users. Financial asset depreciation tracking can also be incorporated, providing hospital management with accurate insights into the current book value of equipment over time. Additionally, an automated email notification system can be implemented to alert administrators when assets have been in use beyond a defined time threshold or when scheduled maintenance is approaching. Finally, graphical reports and interactive dashboards built using Chart.js or similar libraries can be integrated to provide deeper insights into asset utilization patterns, helping management make informed decisions about resource allocation and procurement.

REFERENCES

- S. L. Ting, S. K. Kwok, A. H. C. Tsang, and W. B. Lee, "The study on using passive RFID tags for indoor positioning," *International Journal of Engineering Business Management*, vol. 3, no. 1, pp. 9–15, 2011.
- M. Najafi, S. Rajabi, and R. Roumina, "Hospital asset management using IoT-enabled real-time location systems," *Journal of Medical Systems*, vol. 42, no. 6, pp. 1–10, 2018.
- Django Software Foundation, "Django: The Web Framework for Perfectionists with Deadlines," Official Documentation, 2024. [Online]. Available: <https://docs.djangoproject.com/>.
- Bootstrap Team, "Bootstrap 5: The World's Most Popular Frontend Open Source Toolkit," 2024. [Online]. Available: <https://getbootstrap.com/>.
- Python Software Foundation, "Python 3 Documentation," 2024. [Online]. Available: <https://docs.python.org/3/>.
- W. W. Tsai, Q. Shao, J. Y. Chung, and E. Yu, "Web services and service-oriented architecture for medical device tracking," *IEEE Transactions on Information Technology in Biomedicine*, vol. 10, no. 4, pp. 658–666, 2006.
- R. Oztekin, T. Galambosi, and G. Gungor, "Asset tracking and management in healthcare using web-based systems," *Procedia Computer Science*, vol. 98, pp. 449–454, 2016.
- M. Vijayalakshmi and K. Murugesan, "A study on hospital asset management and the role of information systems in healthcare," *International Journal of Advanced Research in Computer Science*, vol. 8, no. 3, pp. 1234–1238, 2017.

APPENDIX

Appendix A: Sample Code

A.1 Asset Model (models.py)

```
class Asset(models.Model):  
    name = models.CharField(max_length=200)  
    category = models.ForeignKey(Category, on_delete=models.CASCADE)  
    serial_number = models.CharField(max_length=100, unique=True)  
    purchase_date = models.DateField()  
    location = models.CharField(max_length=200)  
    status = models.CharField(max_length=50, choices=[...], default='available')  
    price = models.DecimalField(max_digits=10, decimal_places=2, default=0.00)  
    added_by = models.ForeignKey(User, on_delete=models.SET_NULL, null=True, blank=True)  
    taken_by = models.ForeignKey(User, ..., related_name='taken_assets')  
    taken_at = models.DateTimeField(null=True, blank=True)  
    location_updated_at = models.DateTimeField(auto_now=True)
```

A.2 Dashboard View (views.py)

```
@login_required  
def dashboard(request):  
    total_assets = Asset.objects.count()  
    available = Asset.objects.filter(status='available').count()  
    in_use = Asset.objects.filter(status='in_use').count()
```

```
maintenance = Asset.objects.filter(status='maintenance').count()

summary = Asset.objects.values('name').annotate(
    total=Count('id'),
    available=Count('id', filter=Q(status='available')),
    in_use=Count('id', filter=Q(status='in_use')),
    maintenance=Count('id', filter=Q(status='maintenance')),
)

return render(request, 'assets/dashboard.html', {...})
```

A.3 Add Asset View (views.py)

```
@login_required
def add_asset(request):
    if request.method == 'POST':
        form = AssetForm(request.POST)
        if form.is_valid():
            asset = form.save(commit=False)
            asset.added_by = request.user
            asset.save()
            return redirect('asset_list')
        else:
            form = AssetForm()
    return render(request, 'assets/add_asset.html', {'form': form})
```

A.4 Asset List View (views.py)

```
@login_required
def asset_list(request):
    assets = Asset.objects.all()
    return render(request, 'assets/asset_list.html', {'assets': assets})
```

A.5 Asset List Template (asset_list.html)

```
<h2>Hospital Asset List</h2>
<table border="1">
<tr>
<th>Name</th>
<th>Category</th>
<th>Serial Number</th>
<th>Location</th>
<th>Status</th>
</tr>

{% for asset in assets %}
<tr>
<td>{{ asset.name }}</td>
<td>{{ asset.category }}</td>
<td>{{ asset.serial_number }}</td>
<td>{{ asset.location }}</td>
<td>{{ asset.status }}</td>
</tr>
{% endfor %}

</table>
```

Appendix B: Sample Screenshots

Screenshots of the application are attached below as part of the project submission. Key screens include the Login Page, Dashboard, Asset List Page, Asset Registration Form, Asset Information Page, and Update Location/Status forms.

The screenshot shows the Django administration interface for adding a new user. The top navigation bar includes the Django logo, the title 'Django administration', and user information: 'WELCOME, TESSA_JOSE. VIEW SITE / CHANGE PASSWORD / LOG OUT'. The breadcrumb trail is 'Home > Authentication and Authorization > Users > Add user'. On the left sidebar, the 'Users' link is highlighted under the 'AUTHENTICATION AND AUTHORIZATION' section. The main content area is titled 'Add user' and includes a sub-header 'After you've created a user, you'll be able to edit more user options.' The form contains three main sections: 'Username' with a text input field and a requirement note; 'Password-based authentication' with radio buttons for 'Enabled' (selected) and 'Disabled'; and 'Password' with a text input field and several password strength requirements. A 'Password confirmation' section with a matching text input field is also present. At the bottom, there are three buttons: 'SAVE', 'Save and add another', and 'Save and continue editing'.

Fig A.1: User Registration page

The screenshot displays the login page for the 'Hospital Asset Management' system. The page features a blue header with a hospital icon and the text 'Hospital Asset Management' and 'Login to continue'. Below the header, there are input fields for 'Username' (containing 'Tessa_Jose') and 'Password' (masked with dots). A blue 'Login' button is positioned below the password field. The footer of the login form reads 'Hospital Asset Tracking System'. The background of the page is a blurred image of a stethoscope on a desk. At the very bottom, a copyright notice states: '© 2026 Hospital Asset Management System. All rights reserved.'

Fig A.2: User Login Page

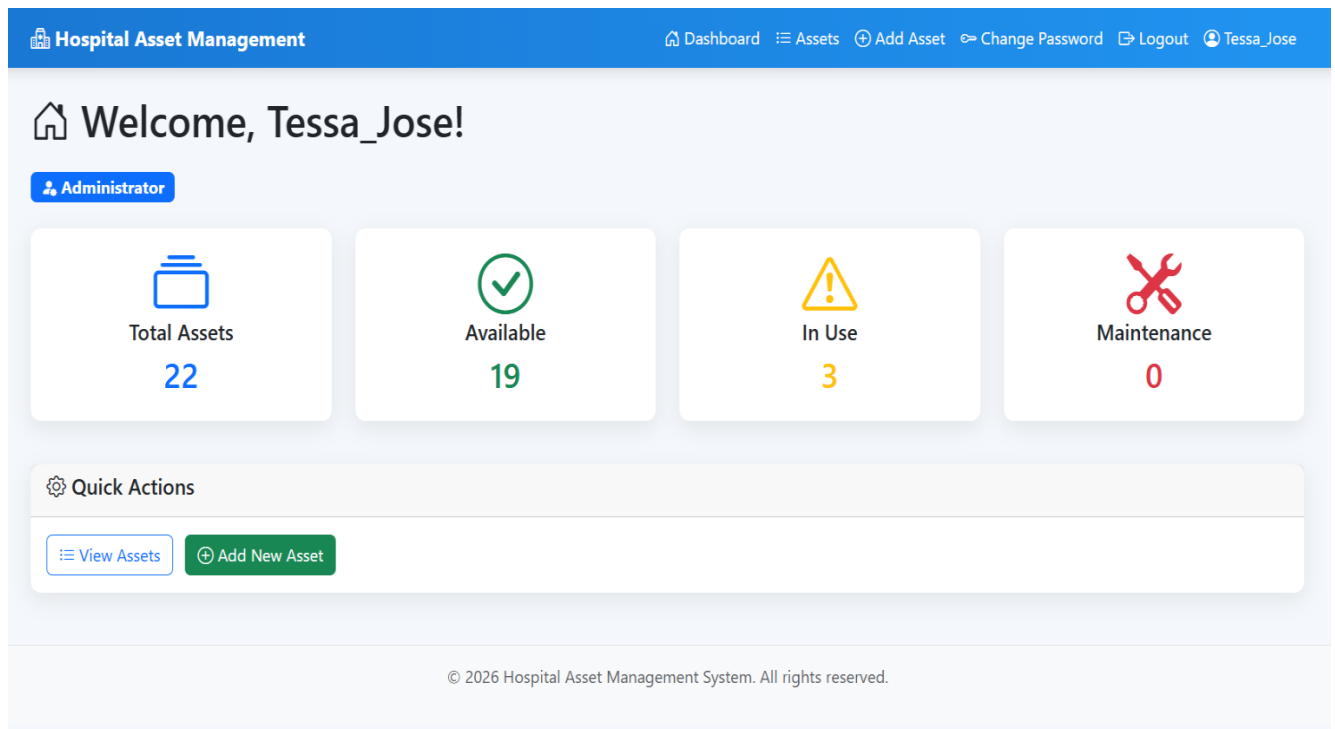


Fig A.3: Admin Dashboard Page

Search assets by name, serial number or location						Search
Asset Details						
Name	Serial Number	Status	Location	Price	Actions	
Oxygen Cylinder	OC-0001	Available	ICU	₹1000.00	Location	Status Info Delete
Oxygen Cylinder	OC-0001-2	Available	ICU	₹1000.00	Location	Status Info Delete
Oxygen Cylinder	OC-0001-3	Available	ICU	₹1000.00	Location	Status Info Delete
Oxygen Cylinder	OC-0001-4	Available	ICU	₹1000.00	Location	Status Info Delete
Defibrillators	DB-0001	Available	ICU	₹50000.00	Location	Status Info Delete
Defibrillators	DB-0001-2	Available	ICU	₹50000.00	Location	Status Info Delete
Defibrillators	DB-0001-3	Available	ICU	₹50000.00	Location	Status Info Delete
Wheelchair	WHL-2034	Available	Emergency Department	₹8000.00	Location	Status Info Delete
Wheelchair	WHL-2034-2	Available	Emergency Department	₹8000.00	Location	Status Info Delete
Wheelchair	WHL-2034-3	Available	Emergency Department	₹8000.00	Location	Status Info Delete
Wheelchair	WHL-2034-4	Available	Emergency Department	₹8000.00	Location	Status Info Delete
Wheelchair	WHL-2034-5	Available	Emergency Department	₹8000.00	Location	Status Info Delete

Fig A.4: Admin Asset View Page

Hospital Asset Management [Dashboard](#) [Assets](#) [Add Asset](#) [Change Password](#) [Logout](#) [Tessa_Jose](#)

Add Asset

Name

Category

Serial number

Purchase date

Location

Status

Price

Quantity

[Save](#) [Back](#)

© 2026 Hospital Asset Management System. All rights reserved.

Fig A.5: Admin Add Asset Page

Hospital Asset Management [Dashboard](#) [Assets](#) [Add Asset](#) [Change Password](#) [Logout](#) [Tessa_Jose](#)

Asset Information

Name	Oxygen Cylinder
Serial Number	OC-0001-2
Status	available
Location	ICU
Category	Medical Equipment
Last Location Update	March 15, 2026, 8:47 a.m.
Purchase Date	March 12, 2026
Price	₹1000.00
Added By	Tessa_Jose

[Back](#)

© 2026 Hospital Asset Management System. All rights reserved.

Fig A.6: Admin Asset Information Page

The screenshot shows the 'Change Password' page of the Hospital Asset Management system. The page has a blue header with the system name and navigation links. The main content area contains a form with three input fields: 'Old Password', 'New Password', and 'Confirm New Password'. Below the fields are two buttons: 'Change Password' (blue) and 'Back to Dashboard' (grey). The footer contains a copyright notice.

Hospital Asset Management | Dashboard | Assets | Add Asset | Change Password | Logout | Tessa_Jose

Change Password

Old Password

New Password

Confirm New Password

[Change Password](#)

[Back to Dashboard](#)

© 2026 Hospital Asset Management System. All rights reserved.

Fig A.7: Admin Password Change Page

The screenshot shows the 'Staff Dashboard' page of the Hospital Asset Management system. The page has a blue header with the system name and navigation links. The main content area features a welcome message, a 'Staff' button, and four summary cards: 'Total Assets' (22), 'Available' (19), 'In Use' (3), and 'Maintenance' (0). Below these cards is an 'Asset Availability Summary' table.

Hospital Asset Management | Dashboard | Assets | Change Password | Logout | Bennet_Jose

Welcome, Bennet_Jose!

[Staff](#)

Total Assets

22

Available

19

In Use

3

Maintenance

0

Asset Availability Summary

Asset Name	Total Quantity	Available	In Use	Maintenance
Defibrillators	3	3	0	0
ECG Machine	2	2	0	0
Infusion Pumps	1	0	1	0
Oxygen Cylinder	4	4	0	0

Fig A.8: Staff Dashboard Page

Search assets by name, serial number or location Search

Asset Details

Name	Serial Number	Status	Location	Price	Actions
Oxygen Cylinder	OC-0001	Available	ICU	₹1000.00	Take Location Status
Oxygen Cylinder	OC-0001-2	Available	ICU	₹1000.00	Take Location Status
Oxygen Cylinder	OC-0001-3	Available	ICU	₹1000.00	Take Location Status
Oxygen Cylinder	OC-0001-4	In Use	ICU	₹1000.00	Return Location Status
Defibrillators	DB-0001	Available	ICU	₹50000.00	Take Location Status
Defibrillators	DB-0001-2	Available	ICU	₹50000.00	Take Location Status
Defibrillators	DB-0001-3	Available	ICU	₹50000.00	Take Location Status
Wheelchair	WHL-2034	Available	Emergency Department	₹8000.00	Take Location Status
Wheelchair	WHL-2034-2	Available	Emergency Department	₹8000.00	Take Location Status

Fig A.9: Staff View Asset Page

Hospital Asset Management Dashboard Assets Change Password Logout Bennet_Jose

Update Status

Status: Available Available In Use Maintenance

© 2026 Hospital Asset Management System. All rights reserved.

Fig A.10: Staff Asset Status Updation Page

The screenshot shows a web application interface for 'Hospital Asset Management'. The top navigation bar is blue and contains the following links: Dashboard, Assets, Change Password (active), Logout, and a user profile for Bennet_Jose. The main content area is light blue and features a white modal box titled 'Change Password'. Inside the modal, there are three input fields: 'Old Password', 'New Password', and 'Confirm New Password'. Below these fields are two buttons: a blue 'Change Password' button and a grey 'Back to Dashboard' button. At the bottom of the page, there is a copyright notice: '© 2026 Hospital Asset Management System. All rights reserved.'

Fig A.11: Staff Password Change Page

The screenshot shows the Django administration login page. The header is a teal bar with the text 'Django administration'. Below the header, a red-bordered box contains a warning message: 'You are authenticated as Bennet_Jose, but are not authorized to access this page. Would you like to login to a different account?'. Below this message, there are two input fields: 'Username:' and 'Password:'. At the bottom, there is a blue 'Log in' button.

Fig A.12: Role based Login Page